NAZARBAYEV UNIVERSITY

Programming for Engineers

Detailed Reading Notes

Week 3: String Manipulation, Guess-and-Check, Approximate Solutions, and Bisection

Yanwei Wang

School of Engineering

Nazarbayev University

Fall 2026

Represent the data clearly ⟶ Search systematically ⟶ Accept a justified approximation ⟶ Verify the result

About these notes

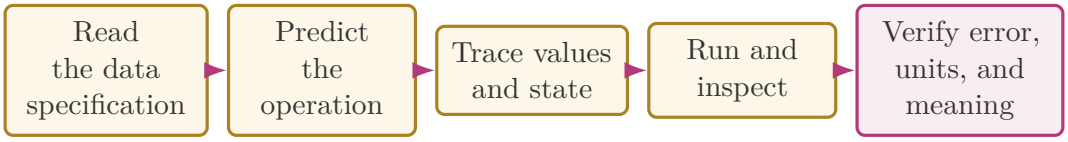
These reading notes expand the complete Week 3 lecture-slide set for *ENG 101 - Programming for Engineers*. The three lectures develop string manipulation for engineering data, controlled guess-and-check searches, approximate numerical solutions, and the bisection method for root finding. The organization, terminology, examples, warnings, activities, and engineering interpretation follow the Week 3 slides. The explanations are expanded so that the document can be read independently as student preparation and review material.

Every executable Python sample includes comments. The comments identify the purpose of variables, the meaning of indexes and slices, the role of a residual or tolerance, the reason a loop progresses, the assumptions behind a numerical search, and the evidence that should be reported. Deliberately faulty programs are also commented so that the intended defect is clear while students still have to diagnose its effect.

Week 3 links two apparently different tasks. First, a program must interpret text without silently changing its engineering meaning. Second, a numerical method must search for a result without pretending that an uncontrolled guess is a trustworthy approximation. In both tasks, a precise data specification, a stopping rule, and explicit evidence are essential.

How to use these notes

Programming and numerical methods become understandable when each example is treated as a small, testable experiment. The recommended Week 3 study cycle is:



For each string-processing example:

1. write the exact characters, including spaces and separators;
2. number the character positions from zero before indexing or slicing;
3. predict the selected character or substring before execution;
4. decide which transformations are safe and which could damage an identifier;
5. test normal, boundary, empty, malformed, and unexpected-unit cases;
6. print both the processed value and evidence that the expected structure was recognized.

For each numerical-search example:

1. state the target condition and, when possible, write it as a residual equal to zero;
2. identify the candidate set or search interval;
3. define the acceptance rule and the physical unit of its tolerance;
4. identify which variable changes and why the change moves toward stopping;
5. include a range bound or maximum-iteration safeguard;
6. distinguish success from controlled failure;
7. report the estimate, residual or interval width, iteration count, units, assumptions, and limitations.

Engineering check

A transformed string may be syntactically convenient but engineeringly wrong if an asset identifier, leading zero, unit, or separator has been altered. A numerical value may be printed to many digits but remain unjustified if the tolerance, bracket, model, or measurement accuracy does not support those digits.

Week 3 structure

Lecture	Main theme	Core ideas
Lecture 7	String manipulation for engineering data	Ordered and immutable strings; positive and negative indexing; slicing; cleaning and inspecting text; splitting and joining records; character tests; validation of identifiers, values, and units.
Lecture 8	Guess-and-check and approximate solutions	Exact versus approximate answers; candidate generation and evaluation; exhaustive enumeration; progress and termination; residuals and tolerances; incremental search; step-size and cost trade-offs; controlled failure.
Lecture 9	The bisection method	Engineering targets as roots; valid brackets; continuity and sign changes; repeated interval halving; invariants; residual- and interval-based stopping; iteration estimates; edge cases; cooling-time application.

Week 3 learning outcomes

By the end of Week 3, you should be able to:

1. treat a string as an ordered, immutable sequence of characters;
2. use positive and negative indexing and explain valid index boundaries;
3. extract substrings using half-open slices and a step value;
4. use common string methods to clean, inspect, split, and join engineering text;
5. normalize human-entered text without erasing meaningful identifiers;
6. convert a numerical field from text only after its intended meaning is established;
7. design representative, boundary, and malformed-record tests;
8. distinguish exact answers from justified numerical approximations;
9. describe guess-and-check as candidate generation, model evaluation, acceptance, and update;
10. explain how a loop makes progress and why it terminates;
11. define a residual and a tolerance with engineering meaning and units;
12. explain the trade-off among step size, resolution, running time, and the risk of missing an acceptable value;
13. add a maximum-iteration safeguard and report success or failure honestly;

14. rewrite an engineering target condition in the form $f(x) = 0$;
15. verify a sign-changing bracket and state the continuity assumption;
16. carry out and trace bisection iterations while preserving the bracket invariant;
17. choose residual- and interval-width stopping criteria appropriately;
18. estimate the number of bisection iterations required;
19. recognize endpoint roots, discontinuities, and roots that do not produce an endpoint sign change;
20. communicate a numerical result with evidence, units, assumptions, and limitations.

Key idea

The central Week 3 message is: approximation is not guessing without discipline. Define the target, preserve the evidence, control the error, check the physics, and report only the precision the evidence supports.

Contents

How to use these notes **3**
Week 3 structure . . . 4
Week 3 learning outcomes . . . 4

1 String Manipulation for Engineering Data **9**
1.1 Why text is engineering data . . . 9
1.2 Retrieval warm-up: predict before running . . . 10
1.3 A string is an ordered sequence . . . 10
1.4 Positive indexing retrieves one character . . . 11
1.5 Negative indexing counts from the end . . . 11
1.6 Slicing extracts a substring . . . 12
1.6.1 Useful slice shortcuts . . . 12
1.6.2 Predict a set of slices . . . 13
1.7 Strings are immutable . . . 13
1.8 String methods return transformed strings . . . 14
1.8.1 Two method-call mistakes . . . 15
1.9 Normalize human-entered text deliberately . . . 15
1.10 Inspect prefixes, suffixes, and membership . . . 16
1.11 Replace, find, and count . . . 16
1.12 Split a structured record into fields . . . 17
1.12.1 Inspect the number of fields before unpacking . . . 18
1.13 Join cleaned fields into a standard label . . . 18
1.14 A loop can inspect every character . . . 19
1.15 Integrated example: clean, parse, validate, and report . . . 20
1.16 Debugging common string mistakes . . . 21
1.17 Testing a text-processing rule . . . 21
1.17.1 Representative cases . . . 21
1.17.2 Boundary and invalid cases . . . 21
1.18 Lecture 7 exit ticket . . . 22

2 Guess-and-Check and Approximate Solutions **24**
2.1 From repetition to search . . . 24
2.2 Exact and approximate answers . . . 25
2.3 Opening problem: finding the square root of 10 . . . 25
2.4 The essential parts of a numerical search . . . 25
2.5 Exhaustive enumeration for an exact integer cube root . . . 26
2.5.1 Hand trace for x = 27 . . . 27
2.6 Progress and termination . . . 27
2.7 Extend the exact search to negative cubes . . . 27
2.8 Debugging a nonterminating search . . . 28
2.9 Approximation requires an acceptance rule . . . 29
2.10 Residual error is not always input error . . . 29
2.10.1 Function-space criterion . . . 29
2.10.2 Input-space criterion . . . 30
2.11 Incremental search for an approximate square root . . . 30
2.12 Report success or controlled failure . . . 31
2.13 Step size controls resolution and cost . . . 31
2.13.1 A search can run correctly and still miss the answer . . . 31

2.14 Use both success and safety conditions . . . 32
2.15 Why exact floating-point equality is fragile . . . 33
2.16 Engineering example: minimum discrete area . . . 33
2.17 When guess-and-check is a good first method . . . 34
2.18 Numerical-search checklist . . . 35
2.19 Lecture 8 exit ticket: diagnose the search design . . . 35

3 The Bisection Method for Engineering Root Finding 37
3.1 Retrieval warm-up: what is wrong with this search? . . . 37
3.2 Use order to discard possibilities . . . 38
3.3 Engineering targets become roots . . . 38
3.4 What is a bracketed root? . . . 39
3.5 Example bracket for x squared minus 10 . . . 39
3.5.1 Which intervals provide sign-changing brackets? . . . 39
3.6 The bisection invariant . . . 40
3.7 One bisection step by hand . . . 40
3.8 Trace the first five midpoint decisions . . . 41
3.8.1 Visualizing repeated halving . . . 41
3.9 Bisection pseudocode . . . 41
3.10 Bisection setup and bracket check . . . 41
3.11 The bisection loop: preserve the bracket . . . 42
3.12 Finish the report: estimate and evidence . . . 43
3.13 Two useful stopping criteria . . . 44
3.13.1 Residual criterion . . . 44
3.13.2 Interval-width criterion . . . 44
3.13.3 Combining criteria . . . 44
3.14 How many iterations will bisection need? . . . 45
3.15 Incremental search versus bisection . . . 45
3.16 Important boundary and failure cases . . . 45
3.16.1 An endpoint is already a root . . . 45
3.16.2 A sign change across a discontinuity . . . 46
3.16.3 A root may exist without an endpoint sign change . . . 46
3.17 Engineering example: safe cooling time . . . 46
3.17.1 Bracket the cooling time before iterating . . . 47
3.17.2 Model and numerical settings . . . 47
3.17.3 Cooling bisection loop . . . 48
3.17.4 Report and interpret the cooling result . . . 48
3.18 Debugging clinic: repair the bisection logic . . . 49
3.19 Bisection method checklist . . . 50
3.20 Lecture 9 exit ticket: projectile impact time . . . 51

4 Week 3 Synthesis and Practice 53
4.1 The Week 3 concept map . . . 53
4.2 Week 3 quick reference . . . 53
4.2.1 Strings . . . 53
4.2.2 Numerical search and bisection . . . 54
4.3 Engineering questions to ask . . . 54
4.4 Glossary . . . 54
4.5 Cumulative practice: string manipulation . . . 55
4.5.1 Practice 1: trace indexes and slices . . . 55
4.5.2 Practice 2: repair a string-cleaning program . . . 56
4.5.3 Practice 3: parse an instrument record . . . 56

4.6 Cumulative practice: guess-and-check . . . 57
4.6.1 Practice 4: exact fourth-power search . . . 57
4.6.2 Practice 5: identify the search specification . . . 58
4.6.3 Practice 6: search-cost reasoning . . . 58
4.7 Cumulative practice: bisection . . . 59
4.7.1 Practice 7: verify a bracket . . . 59
4.7.2 Practice 8: perform two bisection steps . . . 59
4.7.3 Practice 9: complete bisection program . . . 59
4.7.4 Practice 10: distinguish numerical convergence from model validity . . . 60
4.8 A combined text-to-root workflow . . . 60
4.9 Week 3 study checklist . . . 62

Source Basis and Suggested Reading **63**

1 String Manipulation for Engineering Data

Lecture-slide alignment: Lecture 7 of the Week 3 slide set: reading, cleaning, validating, and communicating text without losing its engineering meaning.

Learning outcomes

By the end of this chapter, you should be able to:

1. treat a string as an ordered, immutable sequence;
2. use positive and negative indexing;
3. extract substrings using slicing;
4. clean and inspect text with common string methods;
5. split a simple engineering record and convert its numerical field;
6. join cleaned fields into a standard label;
7. inspect individual characters with a loop;
8. normalize input without erasing identifiers;
9. test representative, boundary, and malformed records;
10. explain why text-processing errors can become engineering errors.

1.1 Why text is engineering data

Programming for engineers is not limited to numerical formulas. Before a numerical calculation begins, a program may need to interpret:

- a component identifier such as `P-204B`;
- a sensor message such as `T-07,23.50,C`;
- a unit label such as `kPa`;
- a file name such as `test_run_07.csv`;
- a material name entered by a user;
- a status message produced by an instrument.

Consider the raw sensor message

```
" T-07 , 23.50 , C "
```

A useful program must determine which sensor produced the reading, which numerical value was measured, and which unit was reported. It must also decide whether surrounding spaces are

meaningful or accidental. This is a data-interpretation task, not merely a cosmetic text-editing task.

Engineering check

Cleaning text should make the information more explicit. It should not silently change a component identity, remove a meaningful leading zero, alter a unit, or combine fields that should remain separate.

1.2 Retrieval warm-up: predict before running

The following example reviews input normalization, Boolean comparison, and iteration from Week 2.

```
# Store a unit label with accidental spaces and mixed capitalization.
raw_unit = " MPa "

# Remove surrounding spaces, then create a lowercase version.
unit = raw_unit.strip().lower()

# Display the cleaned unit.
print(unit)

# Ask whether the normalized unit exactly matches the expected text.
print(unit == "mpa")

# Visit each character of the two-character string in order.
for character in "P7":
    # Display the character used on the current loop pass.
    print(character)
```

Code 1.1: Warm-up: normalize a unit and inspect a short identifier.

`strip()` removes whitespace only from the beginning and end. `lower()` returns a new lowercase string; it does not modify `raw_unit`. The Boolean value printed is `True`. The loop runs twice, first with `character` equal to `"P"` and then to `"7"`.

1.3 A string is an ordered sequence

A string is a sequence of characters with a definite order. For

`"T-07:C"`

the positions are:

Character	T	-	0	7	:	C
Positive index	0	1	2	3	4	5
Negative index	-6	-5	-4	-3	-2	-1

Order matters. `"T-07:C"` and `"C:T-07"` contain similar characters but encode different information. Length also matters. The function `len("T-07:C")` returns 6, so the valid positive indexes are 0 through 5.

Key idea

For a sequence of length n, the first positive index is 0 and the last positive index is $n - 1$.

1.4 Positive indexing retrieves one character

Indexing selects one character using square brackets.

```
# Store an equipment label containing six characters.
label = "T-07:C"

# Retrieve the first character; Python starts counting at zero.
first = label[0]

# Retrieve the second character.
second = label[1]

# Retrieve the final character using its positive index.
unit_code = label[5]

# Display the selected characters in order.
print(first)
print(second)
print(unit_code)
```

Code 1.2: Retrieve selected characters by positive index.

The output is `T`, `-`, and `C`. Direct indexing requires the requested position to exist.

```
# Store a six-character string whose valid positive indexes are 0 through 5.
label = "T-07:C"

# ERROR: index 6 is outside the valid range and raises IndexError.
print(label[6])
```

Code 1.3: A deliberately invalid direct index.

Do not use `len(label)` as the last valid index. The length is the number of characters, while the last valid index is `len(label) - 1`.

1.5 Negative indexing counts from the end

Negative indexes provide a convenient way to select characters relative to the end of a string.

```
# Store a label whose final characters have fixed meanings.
label = "T-07:C"

# Retrieve the last character.
```

```
print(label[-1])

# Retrieve the second-to-last character.
print(label[-2])

# Retrieve the third-to-last character.
print(label[-3])
```

Code 1.4: Retrieve a fixed suffix using negative indexes.

The outputs are `C`, `:`, and `7`. Negative indexing is useful when the suffix position is meaningful even if the full string length varies.

For example, both `"P-7A"` and `"PUMP-107A"` end with the type code `"A"`. In each case, `s[-1]` retrieves the final code without first calculating the length.

1.6 Slicing extracts a substring

A slice selects a range of characters:

`s[start:stop]`

The start index is included, but the stop index is excluded. This is called a **half-open interval**.

```
# Store a record with known field positions.
record = "T-07,23.50,C"

# Select characters at indexes 0, 1, 2, and 3.
sensor_id = record[0:4]

# Select characters at indexes 5 through 9.
value_text = record[5:10]

# Select the character at index 11 by using a one-character slice.
unit = record[11:12]

# Display the three extracted text fields.
print(sensor_id)
print(value_text)
print(unit)
```

Code 1.5: Extract three fields from a fixed-format record using slices.

The outputs are `T-07`, `23.50`, and `C`.

Why exclude the stop index? Two useful consequences follow:

- The length of `s[start:stop]` is normally `stop - start`.
- Adjacent slices can meet without overlapping: `s[0:4]` and `s[4:8]`.

1.6.1 Useful slice shortcuts

A slice can omit start or stop values and can include a step.

Expression	Meaning	Example
`s[:4]`	From the beginning through index 3	`"PUMP"`
`s[5:]`	From index 5 to the end	`"107A"`
`s[-4:]`	Last four characters	`"107A"`
`s[::2]`	Every second character	`"PMP0A"`
`s[::-1]`	Characters in reverse order	`"A701-PMUP"`

Slicing is more forgiving than direct indexing. A slice that extends beyond the end normally stops safely.

```
# Store a short identifier.
code = "P-07"

# Request characters from index 0 up to, but not including, index 100.
# Python returns all available characters without raising IndexError.
print(code[0:100])
```

Code 1.6: A slice can safely request a stop beyond the string length.

1.6.2 Predict a set of slices

```
# Store an eleven-character valve identifier.
code = "VALVE-204B"

# Select the equipment name at indexes 0 through 4.
print(code[:5])

# Select the three-digit number at indexes 6 through 8.
print(code[6:9])

# Select the final character.
print(code[-1])

# Select the final four characters.
print(code[-4:])

# Select indexes 1, 3, 5, and 7; the stop index 8 is excluded.
print(code[1:8:2])
```

Code 1.7: Slice-prediction activity with an equipment code.

The exact outputs are `VALVE`, `204`, `B`, `204B`, and `AV-0`. The stop index is excluded in every slice. For the final expression, indexes 1, 3, 5, and 7 select `A`, `V`, `-`, and `0`.

1.7 Strings are immutable

Strings cannot be changed one character at a time after they have been created. This property is called **immutability**.

```
# Store a lowercase identifier.
```

```
code = "pump-07"

# ERROR: strings are immutable, so an existing character cannot be replaced.
code[0] = "P"
```

Code 1.8: A deliberately invalid attempt to mutate a string.

Python raises a `TypeError`. To obtain corrected text, construct a new string.

```
# Store the original lowercase identifier.
code = "pump-07"

# Build a new string from an uppercase P and the slice from index 1 onward.
corrected = "P" + code[1:]

# Show that the original string remains unchanged.
print(code)

# Display the newly created corrected string.
print(corrected)
```

Code 1.9: Construct a corrected string without changing the original object.

The variable `corrected` is associated with a new string. We could also reassign `code` to a new value, but that would still not mutate the original string object character by character.

Key idea

A variable name may be rebound to a new string. The string object itself remains immutable.

1.8 String methods return transformed strings

A method is an operation associated with an object. Its general form is:

```
object.method(arguments)
```

The dot selects the method, parentheses call it, and the result is returned. Because strings are immutable, methods such as `strip()`, `lower()`, and `upper()` return new strings.

```
# Store text containing surrounding spaces and mixed capitalization.
raw = " Pump-07 "

# Return a string with surrounding whitespace removed.
clean = raw.strip()

# Return a lowercase version of the cleaned string.
lowercase = clean.lower()

# Return an uppercase version of the cleaned string.
uppercase = clean.upper()

# Display the original and all transformed versions.
print(raw)
print(clean)
print(lowercase)
```

```
print(uppercase)
```

Code 1.10: Create several transformed versions of the same raw text.

The original `raw` still contains the surrounding spaces. The transformations are preserved because their return values are assigned to variables.

1.8.1 Two method-call mistakes

```
# Store text that needs cleaning.
raw = " P-07 "

# MISTAKE: strip is referenced but not called because parentheses are missing.
clean = raw.strip

# This prints a method representation, not the cleaned identifier.
print(clean)
```

Code 1.11: Missing parentheses stores the method object instead of calling it.

```
# Store text containing surrounding spaces.
raw = " P-07 "

# MISTAKE: the cleaned string is returned but not stored anywhere.
raw.strip()

# The original variable still refers to text with spaces.
print(repr(raw))
```

Code 1.12: Ignoring a returned string leaves the original variable unchanged.

The repair is usually:

```
# Store the raw identifier.
raw = " P-07 "

# Call strip and reassign the returned cleaned string.
raw = raw.strip()

# repr reveals the exact stored characters, including quotation boundaries.
print(repr(raw))
```

Code 1.13: Store the returned result of a string method.

1.9 Normalize human-entered text deliberately

Human-entered category names often differ only in capitalization or surrounding spaces. Normalization can make comparison consistent.

```
# Ask the user for a material name; input returns a string.
raw_material = input("Material: ")

# Remove surrounding spaces and use lowercase for comparison.
material = raw_material.strip().lower()
```

```
6
7  # Compare the normalized text with the expected material name.
8  if material == "steel":
9      # This branch handles Steel, STEEL, steel, and surrounding spaces.
10     print("Steel selected")
11 else:
12     # This branch handles all other normalized material names.
13     print("Different material")
```

Code 1.14: Normalize a material name before comparison.

Normalization is useful when capitalization does not carry engineering meaning. It must not be applied blindly. `P-007` and `P-7` may identify different assets. Removing leading zeros or punctuation could destroy identity.

Engineering check

Before transforming text, ask which features are accidental presentation and which features are part of the data specification. Case may be irrelevant for a user-entered material category but essential in a password, file path, chemical symbol, or coded identifier.

1.10 Inspect prefixes, suffixes, and membership

Several string operations answer Boolean questions directly.

```
1  # Store a data-file name and a component identifier.
2  file_name = "test_run_07.csv"
3  component = "PUMP-204B"
4
5  # Ask whether the file name ends with the expected extension.
6  print(file_name.endswith(".csv"))
7
8  # Ask whether the component begins with the required equipment prefix.
9  print(component.startswith("PUMP"))
10
11 # Ask whether the number token occurs anywhere in the component identifier.
12 print("204" in component)
13
14 # Ask whether the text TEMP is absent from the component identifier.
15 print("TEMP" not in component)
```

Code 1.15: Check file type, equipment prefix, and text membership.

All four expressions return Boolean values. These checks are case-sensitive by default, so `"pump" in "PUMP-204B"` is `False`.

A prefix or suffix test is often clearer than a manually constructed slice because it expresses the intended question directly.

1.11 Replace, find, and count

The methods `replace()`, `find()`, and `count()` support simple inspection and standardization.

```
1  # Store a record whose fields are separated by vertical bars.
```

```
message = "P-07|245.6|kPa"

# Create a CSV-like version by replacing every vertical bar with a comma.
csv_like = message.replace("|", ",")

# Find the index of the first vertical bar.
separator_index = message.find("|")

# Count how many vertical bars occur in the message.
separator_count = message.count("|")

# Display the transformed text and separator information.
print(csv_like)
print(separator_index)
print(separator_count)
```

Code 1.16: Inspect separators in a pressure message.

`replace()` returns new text. `find()` returns the index of the first occurrence, or `-1` if the target is absent. `count()` returns the number of occurrences.

Do not use the result of `find()` as though it were always a valid location. A result of `-1` means "not found." Test membership or compare the result before using it as an index.

```
# Store a message that may or may not contain a separator.
message = "P-07|245.6|kPa"

# Find the first separator location.
separator_index = message.find("|")

# Use the index only if find did not return -1.
if separator_index != -1:
    # Display the text before the first separator.
    print(message[:separator_index])
else:
    # Report that the expected structure was not found.
    print("Separator not found")
```

Code 1.17: Check a find result before using the location.

1.12 Split a structured record into fields

The method `split(separator)` divides a string at each separator and returns the pieces. In the following fixed-format example, exactly three fields are expected.

```
# Store one raw sensor record with spaces around each field.
raw_record = " T-07 , 23.50 , C "

# Split at commas and assign the three returned pieces to three names.
sensor_id, value_text, unit = raw_record.split(",")

# Remove accidental spaces from the sensor identifier.
sensor_id = sensor_id.strip()
```

```
9
10 # Convert the numerical field from text to a floating-point value.
11 value_c = float(value_text)
12
13 # Remove spaces and standardize the unit to uppercase.
14 unit = unit.strip().upper()
15
16 # Display the interpreted fields.
17 print(sensor_id)
18 print(value_c)
19 print(unit)
```

Code 1.18: Split, clean, convert, and report a sensor record.

The code assumes exactly two commas and therefore exactly three fields. A missing or extra separator causes unpacking to fail. A nonnumeric value causes `float()` to raise a `ValueError`.

Definition

A **structural assumption** describes the expected organization of a record: number of fields, separator character, permitted identifiers, numerical format, and unit vocabulary.

1.12.1 Inspect the number of fields before unpacking

A more defensive Week 3 pattern first stores the pieces and checks their count.

```
1  # Store a record supplied by an instrument or user.
2  raw_record = " T-07 , 23.50 , C "
3
4  # Split the record into a temporary list of text fields.
5  fields = raw_record.split(",")
6
7  # Require exactly three fields before unpacking them.
8  if len(fields) == 3:
9      # Assign the validated field positions to meaningful names.
10     sensor_id, value_text, unit = fields
11     print("Record contains three fields")
12 else:
13     # Report a structural problem instead of pretending the record is valid.
14     print("Malformed record: expected three fields")
```

Code 1.19: Check record structure before assigning individual fields.

Lists are studied formally later. Here, it is enough to recognize that `split()` returns several pieces and that `len(fields)` counts them.

1.13 Join cleaned fields into a standard label

The method `separator.join(parts)` combines several strings with the chosen separator between adjacent parts.

```
1 # Store three separate identifier fields.
2 area = "north"
3 equipment = "pump"
4 number = "007"
```

```

# Create a short list of standardized text parts.
parts = [area.upper(), equipment.upper(), number]

# Join the parts using a hyphen between each pair.
standard_id = "-".join(parts)

# Display the standardized identifier.
print(standard_id)
```

Code 1.20: Create a standardized equipment identifier.

The result is:

```
NORTH-PUMP-007
```

Code 1.21: Standardized identifier.

Notice that the string `"007"` remains text. Converting it to an integer and back to text would remove the leading zeros and change the identifier.

1.14 A loop can inspect every character

A string is iterable, so a `for` loop can examine its characters in order.

```
# Store an equipment identifier containing letters, punctuation, and digits.
component_id = "P-204B"

# Initialize the digit counter before the loop.
digit_count = 0

# Visit each character in the identifier.
for character in component_id:
    # Ask whether the current character is a decimal digit.
    if character.isdigit():
        # Increase the counter only for a digit character.
        digit_count += 1

# Report the total number of digit characters found.
print(f"Digits: {digit_count}")
```

Code 1.22: Count the digits in a component identifier.

Useful character tests include:

Method	Question answered
`character.isdigit()`	Is the character a decimal digit?
`character.isalpha()`	Is it an alphabetic character?
`character.isalnum()`	Is it a letter or digit?
`character.isspace()`	Is it whitespace?

Whether a character pattern is valid is determined by the engineering data specification. Python supplies the tests, but the engineer defines the rule.

1.15 Integrated example: clean, parse, validate, and report

The next program implements the complete text-processing workflow shown in the lecture slides.

```
# Store a raw pressure record with surrounding and internal spaces.
raw = " P-204 , 247.6 , kPa "

# Separate the record into three text fields at the commas.
component, value_text, unit = raw.split(",")

# Remove spaces and standardize the component identifier to uppercase.
component = component.strip().upper()

# Convert the pressure field from text to a floating-point number.
pressure_kpa = float(value_text)

# Remove surrounding spaces from the unit while preserving its displayed case.
unit = unit.strip()

# Check whether the component follows the expected pressure-device prefix rule.
valid_prefix = component.startswith("P-")

# Normalize only for the unit comparison; keep the original unit for reporting.
valid_unit = unit.lower() == "kpa"

# Print a small evidence-based report.
print("PRESSURE RECORD")
print(f"Component: {component}")
print(f"Pressure: {pressure_kpa:.1f} {unit}")
print(f"Prefix valid: {valid_prefix}")
print(f"Unit valid: {valid_unit}")
```

Code 1.23: Interpret and report a pressure record.

The workflow is:

1. separate the fields;
2. clean each field according to its meaning;
3. convert the numerical field;
4. check the expected structure and vocabulary;
5. print the value together with validation evidence.

Engineering check

This program validates only a prefix and unit label. It does not prove that 247.6 kPa is physically plausible, that the sensor is calibrated, or that the component identifier exists in an asset database. Text validation and engineering validation are related but distinct tasks.

1.16 Debugging common string mistakes

Mistake	Faulty idea	Repair question
Missing method call	`clean = raw.strip`	Did you include parentheses?
Ignored return value	Call `raw.strip()` and then continue using `raw`	Where is the transformed string stored?
Invalid index	`code[len(code)]`	What is the last valid index?
Direct mutation	`code[0] = "P"`	How can slicing construct a new string?
Wrong type	`"23.5" + 1.0`	Which conversion is needed before arithmetic?

A useful debugging strategy is to inspect one fact at a time.

```
# Store text containing spaces that may be difficult to see in ordinary output.
raw = " 23.5 "

# repr shows quotation boundaries and escaped characters explicitly.
print(repr(raw))

# type confirms that the value is still text.
print(type(raw))

# Convert only after the expected numeric meaning is established.
value = float(raw)
print(value)
```

Code 1.24: Use repr and type to reveal hidden string details.

1.17 Testing a text-processing rule

One successful record is weak evidence. A text-processing rule should be tested across several categories.

1.17.1 Representative cases

- normal record: `P-204,247.6,kPa`;
- different capitalization: `p-204,247.6,KPA`;
- surrounding spaces;
- multi-digit and leading-zero identifiers.

1.17.2 Boundary and invalid cases

- empty string;
- missing separator;
- extra separator;
- nonnumeric value field;

- unexpected unit;
- empty component field;
- a component ID with an invalid prefix or forbidden character.

Create five versions of the pressure record and predict which line of the integrated program fails or which Boolean validation becomes false. Include at least one record with a leading-zero identifier and verify that the zero is preserved.

1.18 Lecture 7 exit ticket

```python
# Store a record with vertical-bar separators and surrounding spaces.
raw = " t-12 | 88.4 | c "

# Remove only the outer spaces and replace each vertical bar with a comma.
clean = raw.strip().replace("|", ",")

# Display the transformed record.
print(clean)

# Display the first four characters of the transformed record.
print(clean[:4])

# Ask whether the final character is a lowercase c.
print(clean.endswith("c"))
```

Code 1.25: Exit ticket: transform and inspect a short record.

The first output is `t-12 , 88.4 , c`. The spaces immediately beside the former separators remain because `strip()` removes only outer whitespace. The second output is `t-12`. The third output is `True`. One structural assumption is that vertical bars separate three fields. Invalid cases include a missing separator, an extra separator, a nonnumeric middle field, or an unexpected unit.

Section summary

- A string is an ordered, immutable sequence of characters.
- Positive and negative indexes retrieve individual characters.
- Slices use a half-open interval: the start is included and the stop is excluded.
- String methods return new strings; store the returned value when it is needed.
- Normalization should reflect the meaning of each field and must not erase identity.
- `startswith()`, `endswith()`, `in`, `find()`, and `count()` support explicit text checks.
- `split()` separates a structured record; `join()` combines cleaned pieces.
- A loop can inspect every character and apply tests such as `isdigit()`.

- Text-processing code should be tested with representative, boundary, and malformed records.
- A precise-looking numerical result is meaningless if the input record was interpreted incorrectly.

2 Guess-and-Check and Approximate Solutions

Lecture-slide alignment: Lecture 8 of the Week 3 slide set: turning repeated trials into a controlled numerical search with explicit accuracy and stopping rules.

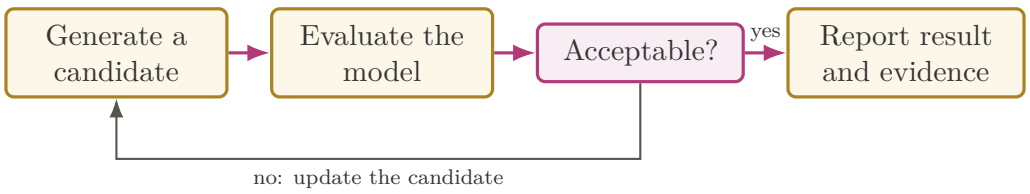
Learning outcomes

By the end of this chapter, you should be able to:

1. distinguish exact analytical answers from approximate numerical answers;
2. describe guess-and-check as candidate generation, model evaluation, and acceptance;
3. implement exhaustive enumeration for a finite integer search;
4. explain how a loop makes progress and why it terminates;
5. define a residual and a tolerance with engineering meaning;
6. distinguish function-space error from input-space error;
7. implement incremental search using a step size;
8. explain the trade-off among step size, accuracy, and running time;
9. add a maximum-iteration safeguard;
10. report success or controlled failure honestly;
11. identify cases where a different search algorithm is needed.

2.1 From repetition to search

A loop becomes a search when each repetition evaluates a candidate and asks whether it satisfies a target condition. The general pattern is:



The word "guess" can be misleading. In a numerical algorithm, a guess should not be an uncontrolled intuition. The program follows a defined sequence of candidates, evaluates each one, and stops according to a stated acceptance and safety rule.

Definition

A **candidate** is a value being tested. An **acceptance rule** states the evidence required to treat a candidate as a solution or useful approximation. An **update rule** determines the next candidate.

2.2 Exact and approximate answers

An exact answer satisfies the mathematical statement without approximation. For example,

$$\sqrt{9} = 3, \qquad 3^2 - 9 = 0.$$

An approximate answer is accepted because its error evidence meets a requirement. For example,

$$\sqrt{10} \approx 3.1623, \qquad \left|3.1623^2 - 10\right| \text{ is small.}$$

Many integer counts and some algebraic calculations are exact within the model. Most measured, simulated, and floating-point engineering quantities are approximate.

Engineering check

More printed digits do not create more truth. Numerical precision should be consistent with the model, input data, units, measurement resolution, and decision being supported.

2.3 Opening problem: how should a computer find $\sqrt{10}$?

The declarative statement

$$x^2 = 10$$

defines the desired relationship, but it does not specify a procedure. An imperative description asks how to generate candidates, evaluate them, and decide when one is close enough.

Candidate g	g^2
3.0	9.00
3.1	9.61
3.2	10.24
3.16	9.9856

The table suggests that the desired value lies near 3.16. A computational method must make the meaning of “near” explicit.

2.4 The essential parts of a numerical search

A complete numerical search specification answers the following questions.

Part	Question	Square-root example
Candidate	What value will be tested?	Current guess g
Evaluation	What model result follows?	g^2 or residual $g^2 - 10$
Acceptance	What counts as good enough?	$\lvert g^2 - 10 \rvert < \varepsilon$
Update	How is the next candidate selected?	$g \leftarrow g + \Delta g$
Stopping safety	What prevents endless work?	Range bound or maximum iterations
Evidence	What is reported?	Candidate, residual, iterations, limitations

A numerical algorithm is incomplete until its candidate set, model, acceptance rule, update, and termination behavior are clear.

2.5 Exhaustive enumeration for an exact integer cube root

Exhaustive enumeration tests candidates systematically until a valid answer is found or the candidate set is exhausted.

```python
# Store the integer whose cube root is being investigated.
x = 125

# Start with the smallest nonnegative integer candidate.
candidate = 0

# Increase the candidate while its cube is still below the target.
while candidate**3 < x:
    # Move to the next integer candidate.
    candidate += 1

# After the loop, check whether the candidate landed exactly on the target.
if candidate**3 == x:
    # Report the exact integer cube root.
    print(f"Cube root = {candidate}")
else:
    # Report that no nonnegative integer candidate was exact.
    print("No integer cube root")
```

Code 2.1: Search for an exact nonnegative integer cube root.

For $x = 125$, candidates 0, 1, 2, 3, 4, and 5 are considered. The loop stops when 5^3 is no longer less than 125, and the exact test succeeds.

2.5.1 Hand trace for $x = 27$

Test number	Candidate	Candidate^3	Loop condition
1	0	0	True
2	1	1	True
3	2	8	True
4	3	27	False

Each pass increases `candidate` by one. When the loop stops, the program checks whether it landed exactly on the target or passed it.

Trace the same algorithm for $x = 20$. The loop stops at candidate 3 because $3^3 = 27$ is not less than 20. The exact test then fails, so the program reports that no integer cube root exists.

2.6 Progress and termination

A loop must not merely repeat; it must make progress toward a stopping condition. For the cube-root search, one progress measure before the target is reached is

$$D = |x| - \text{candidate}^3.$$

As the candidate increases, D decreases toward zero and may become negative.

When inspecting a search loop, ask:

1. Which variable controls the loop condition?
2. Which statement changes that variable?
3. Does the change move toward stopping?
4. Is the stopping state reachable for all intended inputs?
5. What happens for unusual or invalid inputs?

Invariant

For the nonnegative cube-root search, `candidate` is always a nonnegative integer, and it increases by exactly one on each pass. Because a sufficiently large integer cube eventually reaches or exceeds any fixed nonnegative target, the loop terminates.

2.7 Extend the exact search to negative cubes

Negative numbers have real cube roots. A convenient strategy is to search the nonnegative magnitude, then restore the sign for an odd root.

```
# Store a negative perfect cube.
x = -125

```

```python
# Begin the search with a nonnegative candidate.
candidate = 0

# Search using the magnitude so the loop condition is easy to interpret.
magnitude = abs(x)

# Increase the candidate until its cube reaches or exceeds the magnitude.
while candidate**3 < magnitude:
    # Move to the next nonnegative integer candidate.
    candidate += 1

# Check whether the magnitude is an exact integer cube.
if candidate**3 == magnitude:
    # Restore a negative sign when the original target was negative.
    if x < 0:
        candidate = -candidate

    # Report the signed exact root.
    print(f"Integer cube root = {candidate}")
else:
    # Report that no exact integer root exists.
    print("No integer cube root")
```

Code 2.2: Search for an exact integer cube root of a positive or negative integer.

The model knowledge matters: negative numbers have real cube roots, but negative numbers do not have real square roots.

2.8 Debugging a nonterminating search

Consider this deliberately faulty loop.

```python
# Store a positive perfect cube.
x = 64

# Initialize the candidate.
candidate = 0

# The condition is initially true because 0**3 is below 64.
while candidate**3 < x:
    # BUG: this reassignment leaves candidate unchanged at zero.
    candidate = candidate

# This line is never reached because the loop does not terminate.
print(candidate)
```

Code 2.3: Deliberately faulty loop whose candidate never changes.

The smallest repair is `candidate += 1`. A useful diagnostic print would be:

```python
# Store the target and starting candidate.
x = 64
candidate = 0

# Repeat while the cube remains below the target.
while candidate**3 < x:
```

```
    # Display the state before updating it.
    print(f"candidate = {candidate}, cube = {candidate**3}")

    # Correct update: move toward the stopping condition.
    candidate += 1
```

Code 2.4: Print the controlling state to diagnose lack of progress.

Key idea

A progress argument is evidence that an algorithm will finish. It should be part of the design, not an explanation added only after the code happens to work.

2.9 Approximation requires an acceptance rule

For the target equation

$$g^2 = 10,$$

define the residual

$$r(g) = g^2 - 10.$$

An exact root has $r(g) = 0$. A numerical candidate may be accepted when

$$|r(g)| < \varepsilon,$$

where ε is a chosen tolerance.

Definition

A **residual** measures how closely a candidate satisfies the model equation. A **tolerance** specifies the largest residual or uncertainty considered acceptable for the task.

The tolerance must be interpreted in the correct quantity and unit. A temperature residual may be in degrees Celsius; a force-balance residual may be in newtons; a pressure residual may be in kilopascals. For a square-root equation $g^2 - x = 0$, the residual has the units of the squared quantity.

2.10 Residual error is not always input error

Two different error questions should not be confused.

2.10.1 Function-space criterion

$$|f(x)| < \varepsilon_f$$

controls how closely the model output meets the target.

2.10.2 Input-space criterion

$$|x - x^*| < \varepsilon_x$$

controls how close the candidate input x is to the true solution x^*, which is usually unknown.

The same residual can correspond to different input errors when the function is steep or flat. Near a steep crossing, a small change in input may cause a large residual change. Near a flat crossing, a small residual may still correspond to considerable uncertainty in the input.

Engineering check

Choose a stopping criterion that controls the quantity relevant to the engineering decision. A small model-output mismatch is not automatically the same as a sufficiently precise design variable or event time.

2.11 Incremental search for an approximate square root

An incremental search samples candidates separated by a fixed step.

```
# Define the target equation guess**2 = target.
target = 10.0

# Start at the lower end of the candidate range.
guess = 0.0

# Move by one-thousandth on every search step.
step = 0.001

# Accept a candidate when the absolute residual is at most 0.01.
tolerance = 0.01

# Count how many candidate updates have been made.
iterations = 0

# Prevent the search from performing unlimited work.
maximum_iterations = 10_000

# Continue while the residual is too large and work remains available.
while (abs(guess**2 - target) > tolerance
       and iterations < maximum_iterations):
    # Generate the next candidate on the fixed grid.
    guess += step

    # Record one additional search iteration.
    iterations += 1

# Evaluate the final candidate after the loop stops.
residual = abs(guess**2 - target)
```

Code 2.5: Incremental search with a tolerance and work limit.

The loop samples candidate values at spacing $\Delta g = 0.001$. It can stop for one of two reasons:

- success: the residual has become acceptable;

- safety stop: the maximum number of iterations has been reached.

The code must distinguish these outcomes.

2.12 Report success or controlled failure

```python
# Check whether the final residual meets the stated tolerance.
if residual <= tolerance:
    # Report the accepted approximation.
    print(f"Approximation = {guess:.4f}")

    # Report direct evidence of target mismatch.
    print(f"Residual = {residual:.6f}")

    # Report the computational effort.
    print(f"Iterations = {iterations}")
else:
    # State clearly that the final candidate was not accepted.
    print("No acceptable value found")

    # Report the remaining mismatch for diagnosis.
    print(f"Final residual = {residual:.6f}")
```

Code 2.6: Report the candidate only when the acceptance rule is satisfied.

For success, report the candidate, residual, tolerance, and iteration count. For failure, report that the acceptance condition was not met. Do not print the final candidate as though it were a solution merely because the loop stopped.

2.13 Step size controls resolution and cost

A large step produces fewer evaluations, but the candidate sequence may jump over every acceptable value. A small step provides finer resolution, but it requires more evaluations over the same interval.

Suppose a search covers $[a, b]$ with fixed step h. The approximate worst-case number of candidate evaluations is

$$N \approx \frac{b-a}{h}.$$

Reducing h by a factor of 10 usually multiplies the work by about 10 over the same interval.

2.13.1 A search can run correctly and still miss the answer

```python
# Define the square-root target.
target = 10.0

# Start at the beginning of the search interval.
guess = 0.0
```

```
6
7  # Use a relatively coarse step.
8  step = 0.1
9
10 # Require a very small residual.
11 tolerance = 0.001
12
13 # Examine candidates only up to 4.0.
14 while guess <= 4.0:
15     # Test whether the current candidate meets the acceptance rule.
16     if abs(guess**2 - target) < tolerance:
17         print("Found", guess)
18         break
19
20     # Move to the next candidate on the coarse grid.
21     guess += step
```

Code 2.7: A coarse candidate grid can skip the acceptance region.

Near the true root,

$$3.1^2 = 9.61, \qquad 3.2^2 = 10.24.$$

Neither residual is below 0.001. The acceptable interval lies between sampled points. This is a numerical-design issue, not a syntax error.

Do not assume that decreasing the tolerance automatically makes a search more accurate. If the step size remains too large, the method may become less likely to find a candidate satisfying the tighter tolerance.

2.14 Use both success and safety conditions

A robust search often combines an accuracy condition with a finite work budget.

```
1  # Continue only while the result is unacceptable and work remains.
2  while (error > tolerance
3         and iterations < maximum_iterations):
4      # Generate and evaluate the next candidate here.
5      # The actual update depends on the numerical method.
6      candidate += step
7
8      # Count the completed search pass.
9      iterations += 1
```

Code 2.8: Generic two-condition search structure.

The success condition states when the numerical requirement is met. The safety condition guarantees that a poorly designed or unreachable search does not run indefinitely.

Engineering check

Reaching an iteration limit is not a numerical solution. It is a controlled failure that should be communicated and investigated.

2.15 Why exact floating-point equality is fragile

Many decimal fractions cannot be represented exactly in finite binary form.

```
# Start from an exactly representable floating-point zero.
x = 0.0

# Add the approximate floating-point representation of 0.1 ten times.
for _ in range(10):
    x += 0.1

# Display the accumulated floating-point value.
print(x)

# Ask whether its stored representation is exactly equal to 1.0.
print(x == 1.0)

# Ask whether the difference is negligible for this illustrative tolerance.
print(abs(x - 1.0) < 1e-12)
```

Code 2.9: Compare exact equality with a tolerance-based question.

A typical result is:

```
0.9999999999999999
False
True
```

Code 2.10: Typical floating-point result.

The better numerical question is often whether two values are close enough for a justified tolerance, not whether their stored binary representations are identical.

2.16 Engineering example: minimum discrete area

Guess-and-check is especially natural for discrete standard sizes. Suppose a force of 8,000 N acts on a cross-sectional area selected in increments of 5 mm^2. The allowable stress is 150 MPa.

Because

$$1\ \text{N/mm}^2 = 1\ \text{MPa},$$

the stress can be calculated directly as force divided by area in square millimetres.

```
# Store the applied force in newtons.
force_n = 8000.0

# Store the maximum allowable stress in megapascals.
allowable_stress_mpa = 150.0
```

```
6
7  # Start with the smallest candidate area in the design grid.
8  area_mm2 = 10.0
9
10 # Increase the candidate area in standard increments of 5 mm^2.
11 area_step_mm2 = 5.0
12
13 # N/mm^2 is numerically equivalent to MPa.
14 stress_mpa = force_n / area_mm2
15
16 # Continue while the current candidate exceeds the stress limit.
17 while stress_mpa > allowable_stress_mpa:
18     # Move to the next available standard area.
19     area_mm2 += area_step_mm2
20
21     # Recalculate stress for the updated area.
22     stress_mpa = force_n / area_mm2
23
24 # Report the first candidate on the tested grid that passes.
25 print(f"Selected area = {area_mm2:.1f} mm^2")
26 print(f"Stress = {stress_mpa:.2f} MPa")
```

Code 2.11: Select the first tested discrete area that satisfies the stress limit.

The exact threshold from the simplified formula is

$$A = \frac{8000}{150} \approx 53.33 \text{ mm}^2.$$

The tested design grid contains 50 and 55 mm^2, so the first tested safe value is 55 mm^2.

Engineering check

The result is the smallest area in the specified discrete candidate set that satisfies this one stress rule. It is not automatically a manufacturable size, a globally optimal design, or a complete structural safety assessment.

2.17 When guess-and-check is a good first method

Guess-and-check is often useful for:

- small finite candidate sets;
- discrete standard sizes;
- simple prototypes and reference solutions;
- cases where transparency matters more than speed;
- checking a more advanced method independently.

Warning signs include:

- an extremely small step is required;
- the search interval is large;

- the method repeatedly skips acceptable values;
- millions of evaluations are needed;
- the model supplies directional information that the method ignores.

The bisection method introduced in the next lecture uses ordering and sign information to discard half of a valid interval after every evaluation.

2.18 Numerical-search checklist

Before trusting a numerical search, answer all of the following.

1. **Target:** What condition are we trying to satisfy?
2. **Candidate set:** Which values are allowed and physically meaningful?
3. **Model:** What is calculated for each candidate?
4. **Acceptance:** Which residual or error measure is used?
5. **Tolerance:** What error is acceptable, and in which unit?
6. **Update:** How does the next candidate differ?
7. **Progress:** Why does the update move toward stopping?
8. **Safety:** What range or iteration limit prevents endless work?
9. **Evidence:** What candidate, residual, iteration count, and limitation will be reported?

2.19 Lecture 8 exit ticket: diagnose the search design

```
# Define the target equation guess**2 = 2.
target = 2.0

# Start from zero and advance by one-half.
guess = 0.0
step = 0.5

# BUG: exact equality is unreachable on this candidate grid.
while guess**2 != target:
    # Move through 0.0, 0.5, 1.0, 1.5, 2.0, and beyond.
    guess += step

# This statement is not reached because the loop does not terminate.
print(guess)
```

Code 2.12: Deliberately fragile search for the square root of two.

The candidates near $\sqrt{2}$ are 1.0 and 1.5. Neither squared value equals 2 exactly, so the loop continues forever. One repair is:

```
# Define the target, grid spacing, and numerical requirements.
target = 2.0
```

```
guess = 0.0
step = 0.001
tolerance = 0.001
iterations = 0
maximum_iterations = 10_000

# Stop after success or when the finite work budget is exhausted.
while (abs(guess**2 - target) > tolerance
       and iterations < maximum_iterations):
    # Generate the next candidate.
    guess += step
    iterations += 1

# Evaluate the final residual before reporting status.
residual = abs(guess**2 - target)

# Distinguish accepted approximation from controlled failure.
if residual <= tolerance:
    print(f"Approximation = {guess:.4f}")
    print(f"Residual = {residual:.6f}")
else:
    print("Search stopped without an acceptable candidate")
```

Code 2.13: Tolerance and iteration limit repair the search control.

Section summary

- Guess-and-check is a systematic candidate-generation pattern, not random intuition.
- Exact and approximate answers require different acceptance tests.
- Exhaustive enumeration is transparent and useful for small finite or discrete searches.
- A loop needs a progress argument and a reachable stopping condition.
- A residual measures target mismatch; a tolerance defines acceptable mismatch.
- Residual error and input-variable error are not necessarily the same.
- Incremental search uses a fixed step, which controls both resolution and work.
- A coarse step may skip every acceptable candidate; a very fine step may be expensive.
- A maximum-iteration limit converts endless work into reportable controlled failure.
- Floating-point searches should normally use tolerance-based questions rather than exact equality.
- The reported result should include evidence and limitations, not only the final candidate.

3 The Bisection Method for Engineering Root Finding

Lecture-slide alignment: Lecture 9 of the Week 3 slide set: using a valid interval, sign information, and repeated halving to obtain a controlled approximation.

Learning outcomes

By the end of this chapter, you should be able to:

1. rewrite an engineering target condition in the form $f(x) = 0$;
2. explain what it means for a root to be bracketed;
3. state the continuity and sign-change assumptions used by bisection;
4. perform several bisection iterations by hand;
5. implement bisection using variables, conditionals, and a loop;
6. preserve the bracketing invariant when updating endpoints;
7. choose and interpret residual- and interval-based stopping criteria;
8. estimate the number of iterations required;
9. compare incremental search with bisection;
10. recognize invalid brackets, endpoint roots, discontinuities, and roots without sign changes;
11. communicate a numerical result with error evidence and engineering limitations.

3.1 Retrieval warm-up: what is wrong with this search?

```
# Define the square-root target.
target = 10.0

# Start the search at zero.
guess = 0.0

# Use a very small fixed step.
step = 0.000001

# Repeat until the residual falls below one millionth.
# RISK: there is no maximum-iteration or upper-range safeguard.
while abs(guess**2 - target) > 1e-6:
    # Advance by only one millionth on every pass.
    guess += step

# Display the final candidate if the loop eventually stops.
print(guess)
```

Code 3.1: A conceptually simple but extremely inefficient unbounded search.

The method is conceptually capable of approaching the positive root, but it needs roughly 3.16×10^6 candidate updates merely to reach the neighborhood of $\sqrt{10}$. It ignores the directional fact that a squared guess below 10 means the root lies to the right, while a squared guess above 10 means the root lies to the left. It also lacks a finite work limit.

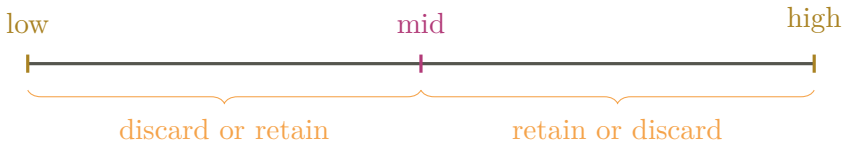
Key idea

Algorithm choice can matter far more than machine speed. Bisection uses mathematical structure to replace millions of tiny steps with a few tens of interval halvings.

3.2 Use order to discard possibilities

When looking for a word in a sorted dictionary, we do not normally inspect every page from the beginning. We open near the middle, decide whether the word must be earlier or later, and discard the impossible half.

The same idea applies to an ordered numerical interval. Evaluate the midpoint, determine which half can still contain the target, and repeat on the retained half.



3.3 Engineering targets become roots

Many engineering questions can be rewritten as equations whose desired condition occurs when a residual is zero.

Engineering question	Root-finding form
When does temperature reach T_{safe}?	$f(t) = T(t) - T_{\text{safe}} = 0$
Which area makes stress equal the allowable value?	$f(A) = \sigma(A) - \sigma_{\text{allow}} = 0$
Which resistance gives a target current?	$f(R) = I(R) - I_{\text{target}} = 0$
When does projectile height reach the ground?	$f(t) = h(t) - 0 = 0$
Which operating point balances generation and loss?	$f(x) = \text{generation}(x) - \text{loss}(x) = 0$

Definition

A **root** of f is a value x^* for which $f(x^*) = 0$. A convenient engineering residual is often "calculated value minus target value."

The sign of the residual can carry useful directional information. For a cooling problem, a positive value of $T(t) - T_{\text{safe}}$ means the component is still above the target; a negative value means it is below the target.

3.4 What is a bracketed root?

An interval $[a, b]$ brackets a root when the available evidence guarantees that at least one root lies inside it. For a continuous function, a common test is

$$f(a)f(b) < 0.$$

The endpoint residuals have opposite signs, so the graph must cross zero somewhere between them.

If $f(a) = 0$ or $f(b) = 0$, an endpoint is already an exact root. A practical validation therefore handles endpoint roots explicitly or uses the non-strict product condition $f(a)f(b) \leq 0$ with care.

Engineering check

A sign change guarantees a root only when the residual is defined and continuous throughout the interval. Domain and model assumptions are part of the algorithm.

3.5 Example bracket for $x^2 - 10 = 0$

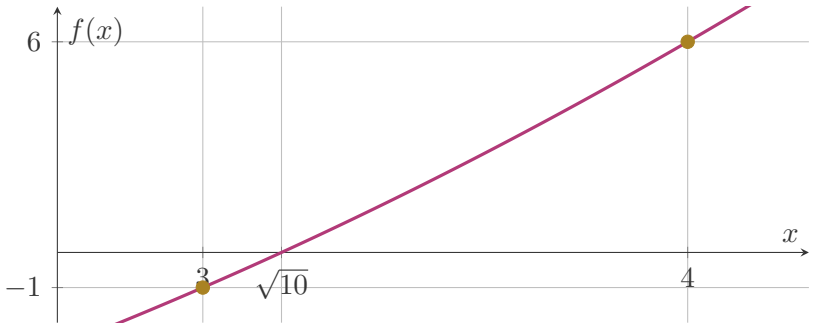
Define

$$f(x) = x^2 - 10.$$

At the endpoints 3 and 4,

$$f(3) = -1, \qquad f(4) = 6, \qquad f(3)f(4) = -6 < 0.$$

Therefore the positive root lies in $[3, 4]$.



3.5.1 Which intervals provide sign-changing brackets?

For $f(x) = x^2 - 10$, consider the following intervals.

Interval	Sign of $f(a)$	Sign of $f(b)$	Valid sign-change bracket?
$[0, 2]$	negative	negative	No
$[2, 4]$	negative	positive	Yes
$[3, 4]$	negative	positive	Yes
$[-4, -3]$	positive	negative	Yes
$[-2, 2]$	negative	negative	No

The final interval contains both 0 and the region between the two roots, but its endpoints have the same sign and it does not provide the evidence required by basic bisection.

3.6 The bisection invariant

Bisection repeatedly preserves one crucial fact:

Invariant

At the start of every iteration, the current interval $[a, b]$ still brackets at least one root by the sign-change evidence being used.

The algorithm is:

1. start with a valid bracket $[a, b]$;
2. compute the midpoint $m = (a + b)/2$;
3. evaluate $f(m)$;
4. retain the half whose endpoints still have opposite signs or include an exact root;
5. repeat until the selected error measure is acceptable.

The endpoint update must be based on sign evidence, not merely on whether the numerical value of the midpoint seems "large" or "small."

3.7 One bisection step by hand

For $f(x) = x^2 - 10$ and $[a, b] = [3, 4]$:

1. Midpoint:
$$m = \frac{3 + 4}{2} = 3.5.$$
2. Midpoint residual:
$$f(3.5) = 3.5^2 - 10 = 2.25 > 0.$$
3. Endpoint signs:
$$f(3) < 0, \qquad f(3.5) > 0.$$
4. New bracket:
$$[a, b] = [3, 3.5].$$

The bracket width falls from 1 to 0.5 after one midpoint evaluation.

3.8 Trace the first five midpoint decisions

Iteration	Low	High	Midpoint and sign	Retained interval
1	3.00000	4.00000	3.50000, +	[3.00000, 3.50000]
2	3.00000	3.50000	3.25000, +	[3.00000, 3.25000]
3	3.00000	3.25000	3.12500, −	[3.12500, 3.25000]
4	3.12500	3.25000	3.18750, +	[3.12500, 3.18750]
5	3.12500	3.18750	3.15625, −	[3.15625, 3.18750]

The root remains bracketed and the interval width is halved every iteration.

3.8.1 Visualizing repeated halving



The method does not need to know the exact root. It needs a valid bracket and a reliable way to decide which subinterval preserves the evidence.

3.9 Bisection pseudocode

A complete high-level algorithm is:

1. Choose `low` and `high` and verify a sign-changing bracket.
2. Accept an endpoint if its residual is already zero.
3. Compute the midpoint and its residual.
4. Retain the half whose endpoints preserve the sign change.
5. Repeat until the interval width or residual meets the tolerance.
6. Report the estimate, residual, final interval, interval width, and iterations.

3.10 Bisection setup and bracket check

The following code begins the square-root example. It is intentionally shown in stages so that each responsibility remains visible.

```
# Define the equation x**2 - target = 0.
target = 10.0

# Choose an interval expected to contain the positive root.
low = 3.0
```

```
high = 4.0

# Stop when the retained interval is no wider than one millionth.
tolerance_x = 1e-6

# Provide a finite safety limit.
maximum_iterations = 100
iterations = 0

# Evaluate the residual at both interval endpoints.
f_low = low**2 - target
f_high = high**2 - target

# Reject an interval whose endpoint residuals have the same nonzero sign.
if f_low * f_high > 0:
    print("Invalid bracket")
```

Code 3.2: Define the problem and evaluate the endpoint evidence.

A production version should check endpoint roots before rejecting or iterating.

```
# Define a simple root-finding problem.
target = 9.0
low = 3.0
high = 5.0

# Evaluate the endpoint residuals.
f_low = low**2 - target
f_high = high**2 - target

# Accept the lower endpoint when it is already an exact root.
if f_low == 0:
    estimate = low
    print(f"Endpoint root = {estimate}")

# Accept the upper endpoint when it is already an exact root.
elif f_high == 0:
    estimate = high
    print(f"Endpoint root = {estimate}")

# Proceed only when a strict sign change is present.
elif f_low * f_high < 0:
    print("Proceed with bisection")

# Otherwise this interval does not provide a basic sign-changing bracket.
else:
    print("No sign-changing bracket")
```

Code 3.3: Handle endpoint roots and the sign-change condition explicitly.

For this example, $f(3) = 0$, so no iteration is needed.

3.11 The bisection loop: preserve the bracket

```
# Continue while the location uncertainty is too large
# and the finite work budget has not been exhausted.
```

```
while (high - low > tolerance_x
       and iterations < maximum_iterations):
    # Compute the midpoint of the current bracket.
    midpoint = (low + high) / 2

    # Evaluate the residual at the midpoint.
    f_mid = midpoint**2 - target

    # Keep the half [low, midpoint] when it preserves a sign change.
    if f_low * f_mid <= 0:
        # Move the upper endpoint to the midpoint.
        high = midpoint

        # Store the matching residual for the new upper endpoint.
        f_high = f_mid
    else:
        # Otherwise keep the half [midpoint, high].
        low = midpoint

        # Store the matching residual for the new lower endpoint.
        f_low = f_mid

    # Record one completed interval-halving step.
    iterations += 1
```

Code 3.4: Bisection loop for the positive square root of ten.

After either update, the endpoint residuals still have opposite signs or one endpoint is a root. Updating the endpoint value without updating its residual would break the program's internal consistency.

Do not compute the midpoint only once before the loop. The midpoint must be recomputed from the updated endpoints on every iteration.

3.12 Finish the report: estimate and evidence

```
# Use the midpoint of the final bracket as the root estimate.
estimate = (low + high) / 2

# Evaluate the target mismatch at the estimate.
residual = estimate**2 - target

# Measure the remaining width of the root-containing interval.
interval_width = high - low

# Report the proposed answer with a useful display precision.
print(f"Estimate = {estimate:.8f}")

# Report the residual in scientific notation.
print(f"Residual = {residual:.3e}")

# Report the location-uncertainty evidence.
print(f"Interval width = {interval_width:.3e}")

```

```
# Report computational effort.
print(f"Iterations = {iterations}")
```

Code 3.5: Calculate and report the final bisection evidence.

Each item answers a different question:

- estimate: What answer is proposed?
- residual: How closely does it satisfy the equation?
- interval width: How narrow is the interval known to contain a root?
- iterations: How much interval-halving work was performed?

3.13 Two useful stopping criteria

3.13.1 Residual criterion

$$|f(m)| \leq \varepsilon_f.$$

This stops when the model output is close to the target. Its limitation is that a small residual does not always imply a small input error when the function is flat.

3.13.2 Interval-width criterion

$$b - a \leq \varepsilon_x.$$

This stops when the interval containing the root is sufficiently narrow. If the root remains bracketed and the midpoint is reported, its distance from at least one bracketed root is no more than

$$\frac{b-a}{2}.$$

3.13.3 Combining criteria

A practical program may require both a narrow interval and an acceptable residual, or may stop when either is sufficient for the stated engineering purpose. The choice must be documented.

```
# Compute the current midpoint and residual before testing convergence.
midpoint = (low + high) / 2
f_mid = midpoint**2 - target

# State two separate numerical requirements.
residual_ok = abs(f_mid) <= tolerance_f
interval_ok = (high - low) <= tolerance_x

# Accept only when both stated requirements are satisfied.
if residual_ok and interval_ok:
    print("Both convergence requirements are satisfied")
```

Code 3.6: Example of combined residual and interval checks.

3.14 How many iterations will bisection need?

If the initial interval width is

$$L_0 = b_0 - a_0,$$

then after n iterations,

$$L_n = \frac{L_0}{2^n}.$$

To ensure $L_n \leq \varepsilon_x$, choose

$$n \geq \left\lceil \log_2 \left(\frac{L_0}{\varepsilon_x} \right) \right\rceil.$$

Starting with width 1 and requiring width at most 10^{-6} needs

$$\left\lceil \log_2(10^6) \right\rceil = 20$$

iterations. This predictable reduction is a major strength of bisection.

3.15 Incremental search versus bisection

Feature	Incremental search on $[3, 4]$	Bisection on $[3, 4]$
Update	Add a fixed step	Halve the retained interval
Information used	Candidate order only	Order plus endpoint signs
Resolution 10^{-6}	About 10^6 samples	20 interval halvings
Risk	May skip the acceptance region	Converges if its assumptions hold
Error evidence	Depends on step and residual	Explicit bracket width
Transparency	Very simple	Still simple but more structured

Bisection is not universally the fastest root solver, but it is reliable, transparent, and easy to verify when a continuous sign-changing bracket is available.

3.16 Important boundary and failure cases

3.16.1 An endpoint is already a root

A strict test `f_low * f_high < 0` would reject a bracket with one endpoint exactly at the root because the product is zero. Check endpoints explicitly before iteration.

3.16.2 A sign change across a discontinuity

Consider

$$f(x) = \frac{1}{x}$$

on $[-1, 1]$. The endpoint values have opposite signs, but f is undefined at zero and has no root. The sign change occurs across a discontinuity.

Discontinuities in engineering models may arise from division by zero, switching rules, contact events, phase changes, invalid domains, or piecewise constitutive laws. A sign change across such a point is not evidence of a physical balance state.

3.16.3 A root may exist without an endpoint sign change

A touching root such as

$$f(x) = (x - 2)^2$$

has positive endpoint values on $[0, 4]$, even though $x = 2$ is a root. The graph touches zero and turns around.

Similarly,

$$g(x) = (x - 1)(x - 3)$$

has two roots inside $[0, 4]$, but the endpoint signs are both positive. An even number of crossings can leave the endpoint signs unchanged.

Key idea

Failure of the endpoint sign-change test does not prove that no root exists. It means the interval does not provide the particular evidence required by basic bisection.

3.17 Engineering example: safe cooling time

A component cools according to

$$T(t) = T_\infty + (T_0 - T_\infty)e^{-kt},$$

with

$$T_\infty = 25\,^\circ\text{C}, \qquad T_0 = 100\,^\circ\text{C}, \qquad k = 0.04\ \text{s}^{-1}.$$

Find when the temperature reaches $40\,^\circ$C. Define

$$f(t) = T(t) - 40.$$

The required time is a root of f.

3.17.1 Bracket the cooling time before iterating

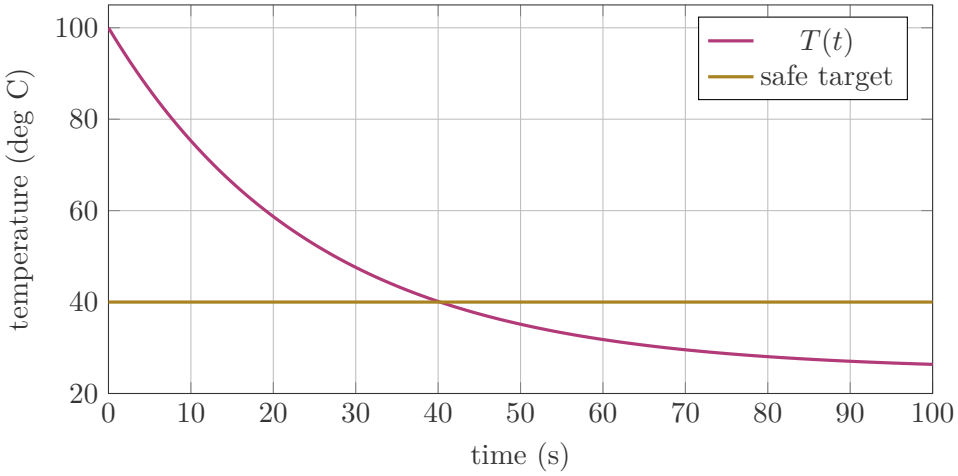
At $t = 0$,

$$T(0) = 100\,^{\circ}\mathrm{C}, \qquad f(0) = 60\,^{\circ}\mathrm{C} > 0.$$

At $t = 100\,\mathrm{s}$,

$$T(100) \approx 26.37\,^{\circ}\mathrm{C}, \qquad f(100) \approx -13.63\,^{\circ}\mathrm{C} < 0.$$

The continuous cooling model therefore changes from above to below the target on $[0, 100]$ seconds.



3.17.2 Model and numerical settings

```python
# Import the exponential function from Python's standard math module.
import math

# Store physical model parameters.
ambient_c = 25.0
initial_c = 100.0
cooling_rate_per_s = 0.04
safe_c = 40.0

# Choose a sign-changing time bracket in seconds.
low_s = 0.0
high_s = 100.0

# Require the final time bracket to be no wider than 0.01 s.
tolerance_s = 0.01

# Provide a finite work limit and initialize its counter.
maximum_iterations = 100
iterations = 0
```

Code 3.7: Define cooling-model parameters and bisection settings.

The interval and its tolerance are in seconds. The temperature residual is in degrees Celsius. The iteration limit prevents endless work if the logic or model assumptions fail.

3.17.3 Cooling bisection loop

```
# Evaluate the residual at the lower endpoint of the bracket.
low_residual = (
    ambient_c
    + (initial_c - ambient_c)
    * math.exp(-cooling_rate_per_s * low_s)
    - safe_c
)

# Halve the time bracket until it meets the time tolerance
# or the maximum number of iterations is reached.
while (high_s - low_s > tolerance_s
       and iterations < maximum_iterations):
    # Compute the midpoint time.
    mid_s = (low_s + high_s) / 2

    # Evaluate temperature minus the safe target at the midpoint.
    mid_residual = (
        ambient_c
        + (initial_c - ambient_c)
        * math.exp(-cooling_rate_per_s * mid_s)
        - safe_c
    )

    # Preserve the sign-changing half of the time interval.
    if low_residual * mid_residual <= 0:
        # The crossing lies between low_s and mid_s.
        high_s = mid_s
    else:
        # The crossing lies between mid_s and high_s.
        low_s = mid_s

        # Update the residual associated with the new lower endpoint.
        low_residual = mid_residual

    # Count one completed bisection iteration.
    iterations += 1
```

Code 3.8: Use bisection to locate the safe-temperature crossing.

Only the residual associated with the changed lower endpoint needs to be retained in this version. If both endpoint residuals are stored, each must always match its current endpoint.

3.17.4 Report and interpret the cooling result

```
# Use the midpoint of the final time bracket as the estimate.
estimate_s = (low_s + high_s) / 2

# Evaluate the model temperature at the estimated time.
estimate_c = (
    ambient_c
    + (initial_c - ambient_c)
    * math.exp(-cooling_rate_per_s * estimate_s)
)

```

```
# Report the estimated crossing time.
print(f"Safe-time estimate = {estimate_s:.2f} s")

# Report the model temperature at that estimate.
print(f"Temperature = {estimate_c:.3f} deg C")

# Report the final time-bracket width.
print(f"Final bracket width = {high_s - low_s:.4f} s")

# Report the number of interval halvings.
print(f"Iterations = {iterations}")
```

Code 3.9: Report the cooling-time estimate and numerical evidence.

The crossing occurs near 40.24 s. The midpoint's location uncertainty relative to a root retained in the bracket is no greater than half the final bracket width.

A direct algebraic rearrangement of the same model provides an independent check:

$$t = -\frac{1}{k} \ln\left(\frac{T_{\text{safe}} - T_\infty}{T_0 - T_\infty}\right) \approx 40.236\,\text{s}.$$

Agreement verifies the implementation for this model; it does not validate the model against real cooling data.

Engineering check

The result is not a complete safety guarantee. It depends on the lumped cooling model, the parameter k, sensor accuracy, ambient temperature, spatial temperature uniformity, and the operational definition of "safe to handle."

3.18 Debugging clinic: repair the bisection logic

```
# Define an initial interval for x**2 - 10 = 0.
low = 3.0
high = 4.0

# BUG: the midpoint is calculated only once.
mid = (low + high) / 2

# Repeat while the bracket is wider than the tolerance.
while high - low > 1e-6:
    # Evaluate the residual at the stale midpoint.
    f_mid = mid**2 - 10

    # BUG: the update directions are reversed for this monotonic example.
    if f_mid < 0:
        high = mid
    else:
        low = mid

    # BUG: mid is never recomputed, so the interval stops changing.

# This line is never reached because the loop becomes infinite.
print(mid)
```

Code 3.10: Deliberately faulty bisection code.

At least three defects are present:

1. the midpoint is not recomputed after endpoint updates;
2. the endpoint update direction does not preserve the correct sign-changing half;
3. endpoint sign evidence is not stored or checked;
4. once one endpoint equals the stale midpoint, the interval no longer shrinks and the loop does not terminate.

A corrected version is:

```
# Define the problem and a valid initial bracket.
target = 10.0
low = 3.0
high = 4.0
f_low = low**2 - target

# Repeatedly halve the bracket until it is sufficiently narrow.
while high - low > 1e-6:
    # Recompute the midpoint on every pass.
    mid = (low + high) / 2
    f_mid = mid**2 - target

    # Retain the half that preserves the sign change.
    if f_low * f_mid <= 0:
        high = mid
    else:
        low = mid
        f_low = f_mid

# Report the midpoint of the final bracket.
print((low + high) / 2)
```

Code 3.11: Corrected bisection logic with explicit sign evidence.

3.19 Bisection method checklist

Before accepting a bisection result, check:

1. **Residual:** Is the engineering target correctly written as $f(x) = 0$?
2. **Domain:** Is f defined throughout the interval?
3. **Continuity:** Is a sign change meaningful for this model?
4. **Bracket:** Have both endpoint residuals been evaluated?
5. **Endpoint roots:** Are exact endpoint solutions handled?
6. **Invariant:** Does every update preserve the root-containing sign-change evidence?

7. **Tolerance:** Does the stopping rule control the quantity that matters?
8. **Safety:** Is there a maximum iteration count?
9. **Evidence:** Are estimate, residual, bracket width, and iterations reported?
10. **Interpretation:** Are units, assumptions, physical reasonableness, and limitations stated?

3.20 Lecture 9 exit ticket: projectile impact time

A projectile-height model is

$$h(t) = 30 + 18t - 4.905t^2.$$

The impact time is the positive root of

$$f(t) = 30 + 18t - 4.905t^2 = 0.$$

A physical bracket is $[0, 10]$ seconds because

$$f(0) = 30 > 0, \qquad f(10) = 30 + 180 - 490.5 = -280.5 < 0.$$

A suitable introductory tolerance might be 0.01 s, provided this resolution is appropriate for the model and application. The first midpoint is 5 s, where

$$f(5) = 30 + 90 - 4.905(25) = -2.625 < 0.$$

Because $f(0) > 0$ and $f(5) < 0$, the first retained bracket is $[0, 5]$ seconds. Continuing bisection gives a positive impact time near 4.914 s.

```
# Define a physical time bracket in seconds.
low_s = 0.0
high_s = 10.0

# Evaluate projectile height at the endpoints.
f_low = 30.0 + 18.0 * low_s - 4.905 * low_s**2
f_high = 30.0 + 18.0 * high_s - 4.905 * high_s**2

# Verify the sign-changing evidence before iteration.
if f_low * f_high < 0:
    print("Impact time is bracketed")
else:
    print("Choose a different physical interval")
```

Code 3.12: Set up the projectile impact problem for bisection.

The numerical method determines where the chosen mathematical model reaches zero. The engineer must still decide whether the model, initial height, initial velocity, gravitational acceleration, neglected air resistance, and meaning of "impact" are appropriate.

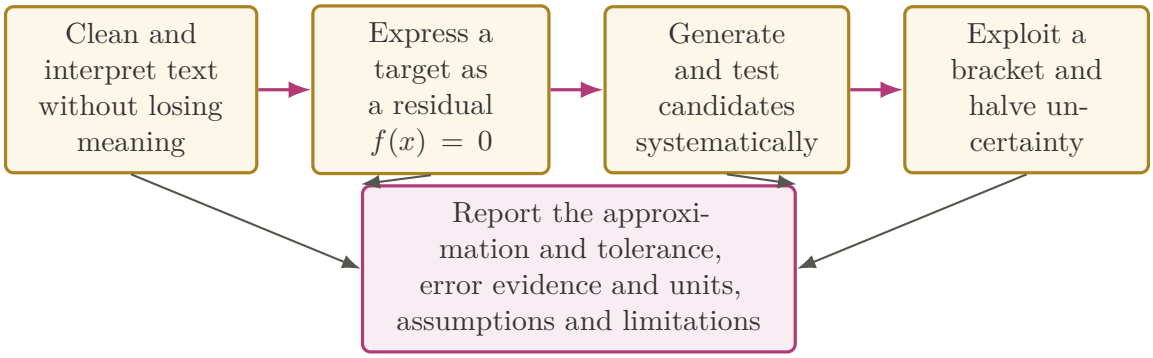
Section summary

- Engineering target conditions can often be written as residual equations $f(x) = 0$.
- For a continuous function, opposite endpoint signs provide a root-containing bracket.
- Bisection computes the midpoint and retains the half that preserves the sign-change evidence.
- The bracket invariant must hold at the start of every iteration.
- The interval width is halved every pass, giving predictable convergence.
- Residual tolerance controls target mismatch; interval tolerance controls location uncertainty.
- A maximum iteration count provides controlled failure protection.
- Endpoint roots, discontinuities, touching roots, and multiple crossings require careful handling.
- Failure of a sign-change test does not prove that no root exists.
- A complete report includes estimate, residual, interval width, iterations, units, assumptions, and limitations.
- Numerical convergence verifies the algorithm's internal requirement; it does not by itself validate the physical model.

4 Week 3 Synthesis and Practice

4.1 The Week 3 concept map

The three lectures form one engineering-computation story:



String manipulation and numerical root finding seem different, but both require explicit specifications. For text, we specify field boundaries, separators, allowed identifiers, and units. For numerical search, we specify candidate values, residuals, tolerances, updates, and stopping conditions. In both cases, silent assumptions create risk.

4.2 Week 3 quick reference

4.2.1 Strings

Expression	Meaning
`s[i]`	Character at index i
`s[a:b]`	Slice from index a through $b-1$
`s[a:b:k]`	Slice using step k
`s.strip()`	Return text without surrounding whitespace
`s.lower()`	Return lowercase text
`s.upper()`	Return uppercase text
`s.startswith(prefix)`	Test whether text begins with a prefix
`s.endswith(suffix)`	Test whether text ends with a suffix
`token in s`	Test whether a token occurs
`s.replace(old,new)`	Return text with replacements
`s.find(token)`	First index, or -1 if absent
`s.count(token)`	Number of occurrences
`s.split(sep)`	Divide text at a separator
`sep.join(parts)`	Combine strings with a separator
`character.isdigit()`	Test whether one character is a digit

4.2.2 Numerical search and bisection

Expression	Meaning
`abs(f(x)) < tolerance`	Residual-based acceptance
`x += step`	Incremental-search candidate update
`iterations += 1`	Count computational work
`iterations < maximum_iterations`	Finite safety condition
`f(a) * f(b) < 0`	Strict sign-changing bracket for continuous f
`mid = (a + b) / 2`	Bisection midpoint
`b - a <= tolerance_x`	Interval-width stopping rule
`abs(f(mid)) <= tolerance_f`	Residual stopping rule
$L_n = L_0/2^n$	Bisection interval width after n halvings

4.3 Engineering questions to ask

For string processing:

- What information does this text encode?
- Which spaces, zeros, separators, and letter cases are meaningful?
- What structure is expected?
- What malformed cases should be rejected?
- Which value and validation evidence should be reported?

For numerical search:

- What target condition are we solving?
- What candidate values or interval are physically valid?
- What residual is measured, and in what unit?
- What tolerance is justified?
- Why must the search terminate?
- What result, error evidence, effort, and limitation should be reported?

4.4 Glossary

Acceptance rule
The criterion used to decide that a candidate is sufficiently accurate or otherwise acceptable.

Bisection
A bracketed root-finding method that halves the retained interval on every iteration.

Bracket
An interval supported by evidence that at least one root lies inside; basic bisection commonly uses opposite endpoint signs and continuity.

Candidate
A trial input being tested by a search algorithm.

Controlled failure
Termination with a clear statement that the numerical acceptance rule was not satisfied.

Exhaustive enumeration
Systematic testing of all candidates in a finite or bounded set.

Half-open slice
A slice that includes its start index but excludes its stop index.

Immutable
Unable to be changed in place after creation; Python strings are immutable.

Incremental search
A search that advances through candidates by a fixed step.

Index
An integer position used to retrieve an element of an ordered sequence.

Invariant
A property deliberately preserved during every iteration of an algorithm.

Iteration count
The number of loop or search updates performed; useful evidence about computational cost.

Normalization
Deliberate transformation of text to a standard representation for comparison or storage.

Residual
The mismatch between a model result and its target, often written $f(x)$.

Root
A value x^* satisfying $f(x^*) = 0$.

Slice
A substring or subsequence selected using start, stop, and optional step indexes.

Step size
The difference between consecutive incremental-search candidates.

String method
An operation associated with a string object, called using dot notation.

Tolerance
A stated upper limit on acceptable numerical mismatch or uncertainty.

Unpacking
Assigning several returned sequence elements to several variable names in one statement.

4.5 Cumulative practice: string manipulation

4.5.1 Practice 1: trace indexes and slices

For the string `"SENSOR-042C"`, predict the following values without running Python:

1. `code[0]`
2. `code[-1]`

3. `code[:6]`
4. `code[7:10]`
5. `code[-4:]`
6. `code[::-1]`

The answers are `"S"`, `"C"`, `"SENSOR"`, `"042"`, `"042C"`, and `"C240-ROSNES"`. The slice stop is excluded, and the leading zero remains because the identifier is text.

4.5.2 Practice 2: repair a string-cleaning program

```
# Store a raw unit label.
raw_unit = " kPa "

# BUG: the method is referenced but not called.
clean_unit = raw_unit.strip

# BUG: the returned lowercase string is ignored.
clean_unit.lower()

# This comparison cannot work as intended because clean_unit is a method object.
print(clean_unit == "kpa")
```

Code 4.1: Deliberately faulty string-cleaning program.

A corrected version calls both methods and stores the returned result.

```
# Store a raw unit label with surrounding spaces.
raw_unit = " kPa "

# Call strip and lower, then store the normalized string.
clean_unit = raw_unit.strip().lower()

# Verify that the normalized value matches the expected unit.
print(clean_unit == "kpa")
```

Code 4.2: Corrected string normalization.

4.5.3 Practice 3: parse an instrument record

Write a program for the record

```
" V-09 | 12.75 | V "
```

that:

1. removes outer whitespace;

2. splits the record at vertical bars;
3. cleans the three fields;
4. converts the measurement to `float`;
5. validates that the identifier begins with `"V-"`;
6. validates that the unit is `"V"`;
7. prints a small report.

```python
# Store the raw instrument record.
raw = "  V-09 | 12.75 | V  "

# Remove outer whitespace before field separation.
clean_record = raw.strip()

# Split at the vertical-bar separators.
component, value_text, unit = clean_record.split("|")

# Clean each field according to its meaning.
component = component.strip().upper()
value_v = float(value_text)
unit = unit.strip().upper()

# Evaluate two explicit structural checks.
valid_component = component.startswith("V-")
valid_unit = unit == "V"

# Print the interpreted value and validation evidence.
print("VOLTAGE RECORD")
print(f"Component: {component}")
print(f"Voltage: {value_v:.2f} {unit}")
print(f"Component valid: {valid_component}")
print(f"Unit valid: {valid_unit}")
```

Code 4.3: One annotated solution for the voltage record.

4.6 Cumulative practice: guess-and-check

4.6.1 Practice 4: exact fourth-power search

Write a program that determines whether 625 has a nonnegative integer fourth root.

```python
# Store the target integer.
target = 625

# Begin with the smallest nonnegative integer candidate.
```

```
candidate = 0

# Move through candidates until the fourth power reaches or exceeds the target.

while candidate**4 < target:
    candidate += 1

# Distinguish an exact fourth root from overshooting the target.
if candidate**4 == target:
    print(f"Integer fourth root = {candidate}")
else:
    print("No nonnegative integer fourth root")
```

Code 4.4: Exhaustive enumeration for an exact fourth root.

The program reports 5 because $5^4 = 625$.

4.6.2 Practice 5: identify the search specification

For an incremental search that solves $x^3 = 50$ on $[0, 5]$ with step 0.001 and residual tolerance 0.01, state:

1. the candidate set;
2. the residual;
3. the acceptance rule;
4. the update;
5. a safety condition;
6. evidence to report.

Candidates are the grid values $0, 0.001, 0.002, \ldots, 5$. The residual is $r(x) = x^3 - 50$. Accept when $|r(x)| \leq 0.01$. Update with `x += 0.001`. Stop at the end of the interval or after a justified maximum iteration count. Report the candidate, residual, tolerance, iteration count, and whether the search succeeded.

4.6.3 Practice 6: search-cost reasoning

How many fixed-step intervals are required to cover $[0, 100]$ with step 10^{-4}?

$$N \approx \frac{100 - 0}{10^{-4}} = 1{,}000{,}000.$$

This estimate is a warning that a more informative algorithm may be preferable.

4.7 Cumulative practice: bisection

4.7.1 Practice 7: verify a bracket

For $f(x) = x^3 - 20$, determine whether each interval provides a sign-changing bracket:

1. $[2, 3]$
2. $[0, 2]$
3. $[-3, 0]$

$f(2) = -12$ and $f(3) = 7$, so $[2, 3]$ is valid. On $[0, 2]$, both endpoint residuals are negative. On $[-3, 0]$, both are negative. Only $[2, 3]$ provides the sign-changing evidence for basic bisection.

4.7.2 Practice 8: perform two bisection steps

Starting from $[2, 3]$ for $f(x) = x^3 - 20$:

1. First midpoint: $m_1 = 2.5$. Since $f(2.5) = 15.625 - 20 = -4.375$, retain $[2.5, 3]$.
2. Second midpoint: $m_2 = 2.75$. Since $f(2.75) = 20.796875 - 20 > 0$, retain $[2.5, 2.75]$.

The width falls from 1 to 0.25 after two iterations.

4.7.3 Practice 9: complete bisection program

```
# Define f(x) = x**3 - 20 on a valid sign-changing bracket.
low = 2.0
high = 3.0
f_low = low**3 - 20.0
f_high = high**3 - 20.0

# State numerical requirements.
tolerance_x = 1e-6
maximum_iterations = 100
iterations = 0

# Verify endpoint roots and the sign-changing bracket.
if f_low == 0:
    estimate = low
elif f_high == 0:
    estimate = high
elif f_low * f_high > 0:
    estimate = None
    print("Invalid bracket")
else:
    # Preserve a root-containing bracket while reducing its width.
    while (high - low > tolerance_x
```

```
            and iterations < maximum_iterations):
        # Compute and evaluate the current midpoint.
        midpoint = (low + high) / 2
        f_mid = midpoint**3 - 20.0

        # Retain the half with opposite endpoint signs.
        if f_low * f_mid <= 0:
            high = midpoint
            f_high = f_mid
        else:
            low = midpoint
            f_low = f_mid

        # Count one completed interval halving.
        iterations += 1

    # Use the midpoint of the final bracket as the estimate.
    estimate = (low + high) / 2

# Report evidence only when a valid estimate exists.
if estimate is not None:
    residual = estimate**3 - 20.0
    print(f"Estimate = {estimate:.8f}")
    print(f"Residual = {residual:.3e}")
    print(f"Bracket width = {high - low:.3e}")
    print(f"Iterations = {iterations}")
```

Code 4.5: Complete bisection program for the cube root of twenty.

4.7.4 Practice 10: distinguish numerical convergence from model validity

A bisection program finds a temperature threshold crossing to within 0.001 s. Does this prove that the actual component reaches the threshold within 0.001 s?

No. The narrow numerical bracket describes the root of the implemented model. Real uncertainty may be dominated by model assumptions, parameter estimation, spatial temperature gradients, sensor error, or changing environmental conditions. Numerical error and model uncertainty must not be confused.

4.8 A combined text-to-root workflow

Real programs often combine Week 3 ideas. Suppose an instrument message provides a cooling-rate parameter:

```
"COOL-01,0.040,1/s"
```

A complete workflow would:

1. split and validate the text fields;
2. convert the numerical rate only after confirming the unit;

3. build the cooling residual;
4. verify a physical time bracket;
5. apply bisection;
6. report the time estimate, bracket width, residual, source record, and assumptions.

```python
# Import the exponential function used by the cooling model.
import math

# Store a parameter record received as text.
raw_record = "COOL-01,0.040,1/s"

# Split and clean the three expected fields.
component, rate_text, unit = raw_record.split(",")
component = component.strip().upper()
unit = unit.strip().lower()

# Validate structure before using the numerical parameter.
valid_component = component.startswith("COOL-")
valid_unit = unit == "1/s"

if valid_component and valid_unit:
    # Convert the validated numerical text.
    cooling_rate_per_s = float(rate_text)

    # Define physical model values and a time bracket.
    ambient_c = 25.0
    initial_c = 100.0
    safe_c = 40.0
    low_s = 0.0
    high_s = 100.0
    f_low = initial_c - safe_c

    # Halve the valid bracket until the time uncertainty is small.
    while high_s - low_s > 0.01:
        mid_s = (low_s + high_s) / 2
        mid_temperature_c = (
            ambient_c
            + (initial_c - ambient_c)
            * math.exp(-cooling_rate_per_s * mid_s)
        )
        f_mid = mid_temperature_c - safe_c

        # Preserve the sign-changing time interval.
        if f_low * f_mid <= 0:
            high_s = mid_s
        else:
            low_s = mid_s
            f_low = f_mid

    # Report the numerical result and source identifier.
    estimate_s = (low_s + high_s) / 2
    print(f"Source component = {component}")
    print(f"Safe-time estimate = {estimate_s:.2f} s")
    print(f"Final bracket width = {high_s - low_s:.4f} s")
else:
```

```
51      # Do not calculate with an unrecognized parameter record.
52      print("Invalid cooling-parameter record")
```

Code 4.6: Compact integration of record parsing and cooling-time bisection.

Engineering check

The combined program is useful only if both stages are trustworthy. Correct root-finding cannot repair a misread unit, and correct string parsing cannot validate an inappropriate physical model.

4.9 Week 3 study checklist

Before moving to Week 4, you should be able to do all of the following without copying a completed solution:

- number string indexes and predict slices;
- explain why strings are immutable;
- use and store the results of common string methods;
- preserve leading zeros and meaningful punctuation in identifiers;
- split a three-field record and convert only its numerical field;
- design malformed-record tests;
- identify candidate, model, acceptance, update, progress, and safety in a search;
- explain why a fixed-step search can miss a root;
- define residual and tolerance with units;
- verify a sign-changing bracket and continuity assumption;
- trace several bisection iterations by hand;
- explain and preserve the bisection invariant;
- calculate the number of interval halvings required for a target width;
- report a numerical estimate with evidence and limitations.

Source Basis and Suggested Reading

These notes were developed to accompany the complete Week 3 lecture-slide set and preserve its organization, terminology, examples, engineering cautions, activities, and level of detail. The principal sources are:

1. Yanwei Wang, *Programming for Engineers - Week 3: String Manipulation, Guess-and-Check, Approximate Solutions, and Bisection*, complete lecture-slide set, School of Engineering, Nazarbayev University, Fall 2026.
2. Nazarbayev University, *ENG 101 - Programming for Engineers: Course Specification Report*, Fall 2026, Session 3.
3. Yanwei Wang, *Programming for Engineers with Python*, working course-manuscript draft, Chapters 4 and 12 and Appendix A.
4. John V. Guttag, *Introduction to Computation and Programming Using Python: With Application to Understanding Data*, 2nd ed., MIT Press, Sections 2.3 and 3.1–3.4.

Primary preparation for students

For every short example:

1. predict the selected characters, generated candidates, or retained interval before execution;
2. trace indexes, search variables, residuals, and endpoint signs by hand;
3. run the program from a clean session;
4. modify one identifier, separator, step, tolerance, or bracket;
5. test representative, boundary, malformed, and failure cases;
6. explain what the program establishes and what remains outside its evidence.

Key idea

Approximation is not guessing without discipline. Define the target. Preserve the bracket. Control the error. Check the physics. Report only the precision the evidence supports.